

17. Manejo básico de excepciones

`TRY`, `EXCEPT` Y ERRORES ESPERABLES EN PYTHON

Programación I
Ingeniería Electrónica y Telecomunicaciones

Comisión 3
Profesor: Lautaro Linquimán
Universidad Nacional de Río Negro



Repaso express

Clase anterior

1. ¿Para qué y porque se utilizan los archivos de tipo `.json`?
2. ¿Para qué usamos `json.dump()` y `json.load()`?
3. ¿Los archivos de tipo `.json` son iguales a un diccionario de Python?

¿Para que sirven las excepciones?

Los programas hablan con el mundo real:

- personas que escriben datos;
- archivos que pueden faltar;
- formatos que pueden venir mal;
- otros sistemas que a veces no funcionan.

Las excepciones nos permiten responder a esos casos sin que el programa termine de golpe.

¿Qué es una excepción?

Una excepción es una señal de que algo salió mal **mientras el programa se estaba ejecutando**.

No siempre es un error de escritura del código.

También puede pasar porque:

- el usuario escribió un dato inesperado;
- falta un archivo;
- intentamos usar una posición que no existe;
- el contenido recibido desde afuera no tiene el formato esperado.

¿Qué hace Python cuando aparece?

Cuando Python encuentra una excepción:

1. corta la ejecución normal del programa;
2. muestra el traceback;
3. muestra la línea y el tipo de excepción;
4. termina el programa si nadie maneja ese caso.

Eso es útil para entender el problema, pero no siempre queremos que el programa se cierre de golpe.

La mayoría de los lenguajes manejan excepciones

La idea aparece en muchos lenguajes modernos.

Lenguaje	Forma habitual
Python	<code>try</code> / <code>except</code>
JavaScript	<code>try</code> / <code>catch</code>
Java / C#	<code>try</code> / <code>catch</code>

Cambia la palabra, no la idea central: intentar una operación y manejar un error esperable.

Rompiendo programas

Calculador de edad

```
edad = input("Edad: ")
edad = int(edad)

print(f"El año que viene vas a tener {edad + 1}")
```

Calculadora de división

```
dividendo = int(input("Dividendo: "))
divisor = int(input("Divisor: "))

print(dividendo / divisor)
```

Estructura básica de control de excepción

```
try:  
    # codigo que hay que cuidar  
    operacion_que_puede_fallar()  
except TipoDeError:  
    # codigo de control de excepcion  
    responder_al_error()
```

```
try:  
    # codigo que hay que cuidar  
    operacion_que_puede_fallar()  
except:  
    # codigo de control de excepcion  
    responder_al_error()
```

`try` contiene la operación riesgosa.

`except` indica qué tipo de excepción vamos a manejar.

try / except

```
texto = input("Cantidad: ")

try:
    cantidad = int(texto)
    print(cantidad)
except ValueError:
    print("ERROR")
```

1. Python intenta ejecutar el bloque `try`.
2. Si aparece `ValueError`, ejecuta ese `except`.
3. El programa puede seguir o terminar con un mensaje más claro.

Tenemos los siguientes datos: `20`, `veinte` y vacío.

En que caso entraríamos en el `except`?

Ejercicio 1

Crear `producto_precio.py`.

El programa pide un producto y su precio separados por punto y coma:

```
Producto y precio: Rosquillas;1200
```

Debe separar el texto, convertir el precio a `float` y mostrar:

```
Rosquillas cuesta $1200.0
```

Probar primero sin `try` / `except`.

Después manejar las excepciones.

Casos para probar: `Rosquillas;1200`, `Rosquillas;gratis`, `Rosquillas`.

Validar y manejar excepciones

No son herramientas competidoras. Son dos estrategias que pueden solaparse.

Mismo caso: convertir texto a `int`.

LBYL: mirar antes de saltar / Look Before You Leap.

Verificar antes con `if`.

```
if texto.isdigit():  
    numero = int(texto)  
else:  
    print("No se pudo convertir.")
```

EAFP: pedir perdón antes que permiso / Easier to Ask for Forgiveness than Permission.

Intentar y capturar si falla.

```
try:  
    numero = int(texto)  
except ValueError:  
    print("No se pudo convertir.")
```

¿Cuándo uso cada una?

`if` para reglas del problema:

- rangos;
- opciones permitidas;
- estados válidos.

`try` / `except` para operaciones que pueden fallar:

- conversiones;
- apertura de archivos;
- acceso a recursos.

Dentro de un programa van a convivir.

Excepción **FileNotFoundError**

La excepción **FileNotFoundError** se lanza cuando no se encuentra un archivo.

Implementemos el siguiente código:

```
nombre_archivo = input("Archivo: ")

with open(nombre_archivo, "r") as archivo:
    contenido = archivo.read()

print(contenido)
```

Probamos:

```
Archivo: datos.txt
Archivo: no_existe.txt
```

Manejando la apertura de un archivo

```
nombre_archivo = input("Archivo: ")

try:
    with open(nombre_archivo, "r") as archivo:
        contenido = archivo.read()

        print(contenido)
except FileNotFoundError:
    print("No se encontro el archivo indicado.")
```

Ejercicio 2

Crear `palabra_por_posicion.py`.

Usen el archivo `frases.txt`.

El programa pide una ruta de archivo y una posición:

```
Archivo: frases.txt  
Posición: 3
```

Debe abrir el archivo, separar las palabras y mostrar la palabra pedida.

Usamos numeración humana: `1` significa la primera palabra.

Agregar el control de excepciones necesarios para que el programa no falle.

Extrayendo mensajes de excepciones

Podemos extraer el mensaje de la excepcion de la siguiente manera:

```
try:
    float("hola")
except ValueError as error:
    print(error)
    print("could not convert string to float" in str(error))
```

Lanzando excepciones

Es posible que lancemos nuestras propias excepciones utilizando la palabra reservada `raise + TipoExcepción`.

Por ejemplo:

```
def sumador(anterior, siguiente):  
    if siguiente < anterior:  
        raise ValueError(f"El numero siguiente no puede ser más chico que el anterior")  
    else:  
        print(siguiente + anterior)
```

Tipos comunes de excepción

Excepción	Suele aparecer cuando...
<code>ValueError</code>	el valor no sirve para una operación
<code>TypeError</code>	usamos un tipo de dato inadecuado
<code>NameError</code>	usamos un nombre que no existe
<code>IndexError</code>	pedimos una posición inexistente
<code>KeyError</code>	pedimos una clave inexistente
<code>FileNotFoundError</code>	intentamos abrir un archivo que no está
<code>ZeroDivisionError</code>	dividimos por cero

No hace falta memorizarlas: hay que leer el nombre en el mensaje de error.